EXP:-06.
13-08-2026

ERROR DETECTION AND CORRECTION USING HAMMING CODE.

AIM:

To write a program to implement error detection and correction using the hamming code.

ALGORITHM:-

1. Start the program.
2. Read the number of 4-bit data blocks (n) forming the input data stream.
3. For each 4-bit data block D=d1 d2 d3 d4, perform Hamming encoding:

   a) Arrange 7 bits with parity bits at positions 1, 2 and 4 and data bits at positions 3, 5, 6 and 7.

   b) Compute the parity bits using even parity.

   P_1 = d1 XOR d2 XOR d4.

   P_2 = d1 XOR d3 XOR d4.

   P_4 = d2 XOR d3 XOR d4.

   c) Form the 7-bits Hamming code word: P1 P2 d1 P4 d2 d3 d4.

4. Simulate transmission by introducing a single bit error code at a chosen bit position.
5. At the reciever, recompute the check bits from the recieved code word.
6. Compute error position = C4 C2 C1
7. If error position = 0, no error has occurred.
   Else, the bit at "Error Position" is inverted to correct the error

8. Extract the original 4 data bits from the corrected 7-bit code word.

9. Repeat steps 3 to 8 for all n blocks in the data stream.

10. Stop the stream.

PROGRAM:-

```
def encode(d):
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2].
    h[6] = d[3].
    h[0] = h[2] ^ h[4] ^ h[6].
    h[1] = h[2] ^ h[5] ^ h[6].
    h[3] = h[4] ^ h[5] ^ h[6].

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6].
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6].

    error_pos = c1*1 + c2*2 + c4*4.
    if error_pos != 0:
        print(f" Error detected at bit position : {error_pos}
                                    -> Correcting...").

        h[error_pos - 1] ^= 1.
    else:
        print ("No error detected").
    return error_pos
```

```
def to_str(bits):
    return "".join(str(b) for b in bits).

def main():
    print("=== HAMMING CODE - ERROR DETECTION AND
                    CORRECTION ===\n")
    n = int(input("Enter number of 4-bit data blocks
                    in the stream: ")).

    for b in range(1, n+1):
        print(f"\n--- Block {b} ---").
        d = list(map(int, input("Enter 4 data bits
                (space separated, eg: 1 0 1 1): ").
                    split())).

        h = encode(d).
        print("Original Data Bits : {to_str(d)}").

        pos = int(input("Enter bit position to introduce
        error (1-7, 0 = no error): ")).

        if 1 <= pos <= 7:
            h[pos-1] ^= 1.
            print(f"Transmitted Code (with error induced at
            pos {pos}): {to_str(h)}").

        else:
            print(f"Transmitted Code (no error induced
                    : {to_str(h)}")

        print(f"Received Code : {to_str(h)}")
        print("Checking for errors....")
        detect_and_correct(h)
```

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 3

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 1 1 0
Original Data Bits          : 1110
Encoded Hamming Code (7 bits): 0010110
Enter bit position to introduce error (1-7, 0 = no error): 4
Transmitted Code (with error induced at pos 4): 0011110
Received Code               : 0011110
Checking for errors...
   Error detected at bit position: 4 -> Correcting...
Corrected Code              : 0010110
Recovered Data Bits         : 1110

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 0 1 0
Original Data Bits          : 0010
Encoded Hamming Code (7 bits): 0101010
Enter bit position to introduce error (1-7, 0 = no error): 2
Transmitted Code (with error induced at pos 2): 0001010
Received Code               : 0001010
Checking for errors...
   Error detected at bit position: 2 -> Correcting...
Corrected Code              : 0101010
Recovered Data Bits         : 0010

--- Block 3 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 1 0 1
Original Data Bits          : 1101
Encoded Hamming Code (7 bits): 1010101
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1010111
Received Code               : 1010111
Checking for errors...
   Error detected at bit position: 6 -> Correcting...
Corrected Code              : 1010101
Recovered Data Bits         : 1101
```

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===

Enter number of 4-bit data blocks in the stream: 2

--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits          : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0100011
Received Code               : 0100011
Checking for errors...
   Error detected at bit position: 3 -> Correcting...
Corrected Code              : 0110011
Recovered Data Bits         : 1011

--- Block 2 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 0 1 1 0
Original Data Bits          : 0110
Encoded Hamming Code (7 bits): 1100110
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 1100100
Received Code               : 1100100
Checking for errors...
   Error detected at bit position: 6 -> Correcting...
Corrected Code              : 1100110
Recovered Data Bits         : 0110
```

```
print(f"Corrected code : {to_str(h)}").
rd = [h[2], h[4], h[5], h[6]].
print(f"Recovered Data Bits : {to_str(rd)}").

if __name__ == "__main__":
    main().
```

RESULT:-

The program to implement error detection and correction using the hamming code concept was successfully written in python, executed and verified using a sample input data stream.